

CS324

Lectures / Modeling

We started this course by analyzing a language model as a black box:

$$p(x_1, \dots, x_L) \quad \text{or} \quad \text{prompt} \rightsquigarrow \text{completion}$$

Then we looked at the training data of large language models (e.g., The Pile):

$$\text{training data} \Rightarrow p.$$

In this lecture, we will open up the onion all the way and talk about how large language models are built.

Today's lecture will focus on two topics, **tokenization** and **model architecture**.

- 1 **Tokenization**: how a string is split into tokens.
- 2 **Model architecture**: We will discuss mostly the Transformer architecture, which is the modeling innovation that really enabled large language models.

Tokenization

Recall that a language model p is a probability distribution over a **sequence of tokens** where each token comes from some vocabulary $\mathcal{V}$:

[the, mouse, ate, the, cheese]

However, natural language doesn't come as a sequence of tokens, but as just a string (concretely, sequence of Unicode characters):

the mouse ate the cheese

CS324

A **tokenizer** converts any string into a sequence of tokens.

the mouse ate the cheese $\Rightarrow$ [the, mouse, ate, the, cheese]

This is not necessarily the most glamorous part of language modeling, but plays a really important role in determining how well a model will work.

Split by spaces

The simplest solution is to do:

```
text.split(' ')
```

- This doesn't work for languages such as Chinese, where sentences are written without spaces between words:

我今天去了商店。 [gloss: *I went to the store.*]

- Then there are languages like German that have long compound words (e.g., *Abwasserbehandlungsanlage*).
- Even in English, there are hyphenated words (e.g., *father-in-law*) and contractions (e.g., *don't*), which should get split up. For example, the Penn Treebank splits *don't* into *do* and *n't*, a linguistically-informed but not obvious choice.

Therefore, splitting by spaces by spaces to identify words is quite problematic.

What makes a good tokenization?

- We don't want too **many** tokens (extreme: characters or bytes), or else the sequence becomes difficult to model.

CS324

- We don't want too **few** tokens, or else there won't be parameter sharing between words (e.g., should *mother-in-law* and *father-in-law* be completely different)? This is especially problematic for morphologically rich languages (e.g., Arabic, Turkish, etc.).
- Each token should be a linguistically or statistically meaningful unit.

Byte pair encoding

Sennrich et al, 2015 applied the **byte pair encoding** (BPE) algorithm, originally developed for data compression, to produce one of the most commonly used tokenizers.

Learning the tokenizer. Intuition: start with each character as its own token and combine tokens that co-occur a lot.

- Input: a training corpus (sequence of characters).
- Initialize the vocabulary $\mathcal{V}$ be the set of characters.
- While we want to still grow $\mathcal{V}$:
 - Find the pair of elements $x, x' \in \mathcal{V}$ that co-occur the most number of times.
 - Replace all occurrences of x, x' with a new symbol xx' .
 - Add xx' to $\mathcal{V}$.

Example:

- 1 [t, h, e, ␣, c, a, r], [t, h, e, ␣, c, a, t], [t, h, e, ␣, r, a, t]
- 2 [th, e, ␣, c, a, r], [th, e, ␣, c, a, t], [th, e, ␣, r, a, t] (*th* occurs 3x)
- 3 [the, ␣, c, a, r], [the, ␣, c, a, t], [the, ␣, r, a, t] (*the* occurs 3x)
- 4 [the, ␣, ca, r], [the, ␣, ca, t], [the, ␣, r, a, t] (*ca* occurs 2x)

The output of learning is:

CS324

- Updated vocabulary $\mathcal{V}$: [a, c, e, h, t, r, ca, th, the]
- The merges that we made (important for applying the tokenizer):
 - $t, h \Rightarrow th$
 - $th, e \Rightarrow the$
 - $c, a \Rightarrow ca$

Applying the tokenizer. To tokenize a new string, apply the merges in the same order:

- [t, h, e, $_$ o, x]
- [th, e, $_$ o, x]
- [the, $_$ o, x]

Unicode.

- One problem is that (especially in the multilingual setting), there are a lot (144,697) of Unicode characters.
- We certainly will not see all characters in the training data.
- In order to reduce data sparsity even further, we can run BPE on bytes instead of Unicode characters ([Wang et al. 2019](#)).
- Example in Chinese:

今天 [gloss: *today*]
[x62, x11, 4e, ca]

Unigram model (SentencePiece)

Rather than just splitting by frequency, a more “principled” approach is to define an objective function that captures what a good tokenization looks like. We now describe the **unigram model** ([Kudo 2018](#)).

CS324

- It was of the tokenizations supported in the **SentencePiece** tool (Kudo & Richardson, 2018), along with BPE.
- It was used to train T5 and Gopher.

Given a sequence $x_{1:L}$, a tokenization T is a set of

$$p(x_{1:L}) = \prod_{(i,j) \in T} p(x_{i:j}).$$

Example:

- Training data (string): ababc
- Tokenization $T = \{(1, 2), (3, 4), (5, 5)\}$ ($\mathcal{V} = \{\text{ab}, \text{c}\}$)
- Likelihood: $p(x_{1:L}) = \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{4}{9}$.

Algorithm:

- Start with a “reasonably big” seed vocabulary $\mathcal{V}$.
- Repeat:
 - Given $\mathcal{V}$, optimize $p(x)$ and T using the EM algorithm.
 - Compute $\text{loss}(x)$ for each token $x \in \mathcal{V}$ capturing how much the likelihood would be reduced if x were removed from $\mathcal{V}$.
 - Sort by loss and keep the top 80% tokens in $\mathcal{V}$.

Comparing tokenizers

- GPT-2 and GPT-3 used BPE, vocabulary size of 50K
- **Jurassic** used SentencePiece with vocabulary size of 256K

Impact:

CS324

- Given the same string, Jurassic requires 28% fewer tokens than GPT-3, so it is 1.4x faster
- Both Jurassic and GPT-3 use the same context size (2048), so one can feed in 39% more text into the prompt.

Examples of tokenizations for both GPT-3 and Jurassic ([demo](#)):

- GPT-3: [Ab, raham, _Lincoln, _lived, _at, _the, _White, _House, .]
- Jurassic: [Abraham_Lincoln, _lived, _at_the_White_House, .]

Models

Thus far, we have defined language models as a probability distribution over sequences of tokens $p(x_1, \dots, x_L)$, which as we saw was very elegant and powerful (via prompting, a language model can in principle do anything, as GPT-3 hints at). In practice, however, it can be more efficient for specialized tasks to avoid having to generatively model the entire sequence.

Contextual embeddings. As a prerequisite, the main key development is to associate a sequence of tokens with a corresponding sequence of contextual embeddings:

$$[\text{the, mouse, ate, the, cheese}] \xRightarrow{\phi} \left[\begin{pmatrix} 1 \\ 0.1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -0.1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix} \right].$$

- As the name suggests, the contextual embedding of a token depends on its context (surrounding words); for example, consider the.
- Notation: We will $\phi : \mathcal{V}^L \rightarrow \mathbb{R}^{d \times L}$ to be the embedding function (analogous to a feature map for sequences).
- For a token sequence $x_{1:L} = [x_1, \dots, x_L]$, ϕ produces contextual embeddings $\phi(x_{1:L})$.

Types of language models

CS324

We will broaden our notion of language models to three types of models.

Encoder-only (BERT, RoBERTa, etc.). These language models produce contextual embeddings but cannot be used directly to generate text.

$$x_{1:L} \Rightarrow \phi(x_{1:L}).$$

These contextual embeddings are generally used for classification tasks (sometimes boldly called natural language understanding tasks).

- Example: sentiment classification

$$[[\text{CLS}], \text{the, movie, was, great}] \Rightarrow \text{positive.}$$

- Example: natural language inference

$$[[\text{CLS}], \text{all, animals, breathe, [SEP], cats, breathe}] \Rightarrow \text{entailment.}$$

- Pro: contextual embedding for x_i can depend **bidirectionally** on both the left context ($x_{1:i-1}$) and the right context ($x_{i+1:L}$).
- Con: cannot naturally **generate** completions.
- Con: requires more **ad-hoc training** objectives (masked language modeling).

Decoder-only (GPT-2, GPT-3, etc.). These are our standard autoregressive language models, which given a prompt $x_{1:i}$ produces both contextual embeddings and a distribution over next tokens x_{i+1} (and recursively, over the entire completion $x_{i+1:L}$).

$$x_{1:i} \Rightarrow \phi(x_{1:i}), p(x_{i+1} \mid x_{1:i}).$$

- Example: text autocomplete

$$[[\text{CLS}], \text{the, movie, was}] \Rightarrow \text{great}$$

CS324

- Con: contextual embedding for x_i can only depend **unidirectionally** on both the left context ($x_{1:i-1}$).
- Pro: can naturally **generate** completions.
- Pro: **simple training** objective (maximum likelihood).

Encoder-decoder (BART, T5, etc.). These models in some ways can the best of both worlds: they can use bidirectional contextual embeddings for the input $x_{1:L}$ and can generate the output $y_{1:L}$.

$$x_{1:L} \Rightarrow \phi(x_{1:L}), p(y_{1:L} \mid \phi(x_{1:L})).$$

- Example: table-to-text generation

$$[\text{name, :, Clowns, |, eatType, :, coffee, shop}] \Rightarrow [\text{Clowns, is, a, coffee, shop}].$$

- Pro: contextual embedding for x_i can depend **bidirectionally** on both the left context ($x_{1:i-1}$) and the right context ($x_{i+1:L}$).
- Pro: can naturally **generate** outputs.
- Con: requires more **ad-hoc training** objectives.

Preliminaries

We now describe the innards of the embedding function $\phi : \mathcal{V}^L \rightarrow \mathbb{R}^{d \times L}$:

$$[\text{the, mouse, ate, the, cheese}] \xRightarrow{\phi} \left[\begin{pmatrix} 1 \\ 0.1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ -0.1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix} \right].$$

We now introduce the model architectures for language model, with an emphasis on the ubiquitous Transformer architecture. Our exposition of the Transformer architecture will be

CS324

based on these [slides from CS221 on differentiable programming](#), and will depart a bit from the standard presentation.

The beauty of deep learning is being able to create building blocks, just like we build whole programs out of functions. So we want to be able to functions like the following to encapsulate the complexity:

$$\text{TransformerBlock}(x_{1:L}).$$

This function will have parameters which we will include in the body but elide in the function signature for simplicity.

In what follows, we will define a library of building blocks until we get to the full Transformer.

Preliminaries

First, we have to convert sequences of tokens into sequences of vectors. `EmbedToken` does exactly this by looking up each token in an embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ (a parameter that will be learned from data):

[the, mouse, ate, the, cheese]

$$\text{def EmbedToken}(x_{1:L} : \mathcal{V}^L) \rightarrow \mathbb{R}^{d \times L}:$$

- Turns each token x_i in the sequence $x_{1:L}$ into a vector.
- Return $[E_{x_1}, \dots, E_{x_L}]$.

These are exactly the (context-independent) word embeddings of yore. We define an abstract `SequenceModel` function that takes these **context-independent embeddings** and maps them into **contextual embeddings**.

$$\text{def SequenceModel}(x_{1:L} : \mathbb{R}^{d \times L}) \rightarrow \mathbb{R}^{d \times L}:$$

CS324

- *Process each element x_i in the sequence $x_{1:L}$ with respect to other elements.*
- [abstract implementation (e.g., FeedForwardSequenceModel, SequenceRNN, TransformerBlock)]

The simplest type of sequence model is based on feedforward networks (Bengio et al., 2003) applied to a **fixed length** context, just as in an n-gram model:

def FeedForwardSequenceModel($x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Process each element x_i in the sequence $x_{1:L}$ by looking at the last n elements..*
- For each $i = 1, \dots, L$:
 - Compute $h_i = \text{FeedForward}(x_{i-n+1}, \dots, x_i)$.
- Return $[h_1, \dots, h_L]$.

Recurrent neural networks

The first “real” sequence model is a recurrent neural network (RNN), which is a family of models that include simple RNNs, LSTMs, and GRUs. The basic form of an RNN simply computes a sequence of **hidden states** recursively.

def SequenceRNN($x : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Process the sequence $x_1, \dots, x_L$ left-to-right and recursively compute vectors $h_1, \dots, h_L$.*
- For $i = 1, \dots, L$:
 - Compute $h_i = \text{RNN}(h_{i-1}, x_i)$.
- Return $[h_1, \dots, h_L]$.

The actual module that does the hard work is the RNN, which analogous to a finite state machine, takes the current state h , a new observation x , and returns the updated state:

CS324

def RNN($h : \mathbb{R}^d, x : \mathbb{R}^d$) $\rightarrow \mathbb{R}^d$:

- *Updates the hidden state h based on a new observation x .*
- [abstract implementation (e.g., SimpleRNN, LSTM, GRU)]

There are three ways to implement the RNN. The earliest RNN is a simple RNN [Elman, 1990](#), which takes a linear combination of h and x and pushes it through an elementwise non-linear function σ (e.g., logistic $\sigma(z) = (1 + e^{-z})^{-1}$ or more the modern ReLU $\sigma(z) = \max(0, z)$).

def SimpleRNN($h : \mathbb{R}^d, x : \mathbb{R}^d$) $\rightarrow \mathbb{R}^d$:

- *Updates the hidden state h based on a new observation x by simple linear transformation and non-linearity.*
- Return $\sigma(Uh + Vx + b)$.

As defined RNNs only depend on the past, but we can them depend on the future too by running another RNN backwards. These models were used by [ELMo](#) and [ULMFiT](#).

def BidirectionalSequenceRNN($x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{2d \times L}$:

- *Process the sequence both left-to-right and right-to-left.*
- Compute left-to-right: $[h_1^{\rightarrow}, \dots, h_L^{\rightarrow}] \leftarrow \text{SequenceRNN}(x_1, \dots, x_L)$.
- Compute right-to-left: $[h_L^{\leftarrow}, \dots, h_1^{\leftarrow}] \leftarrow \text{SequenceRNN}(x_L, \dots, x_1)$.
- Return $[h_1^{\rightarrow} h_1^{\leftarrow}, \dots, h_L^{\rightarrow} h_L^{\leftarrow}]$.

Notes:

- The simple RNN is difficult to train due to vanishing gradients.
- The Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) (both of RNN) have been developed to address these.

CS324

- Still, even though the embedding h_{200} can depend arbitrarily far back (e.g., on x_1), it is unlikely to depend on it in a “crisp” way (see [Khandelwal et al., 2018](#) for more discussion).
- LSTMs in some sense were really what brought deep learning into full swing within NLP.

We will not discuss these models in the interest of time.

Transformers

Now, we will discuss Transformers ([Vaswani et al. 2017](#)), the sequence model that is really responsible for the takeoff of large language models; they are the building blocks of decoder-only (GPT-2, GPT-3), encoder-only (BERT, RoBERTa), and decoder-encoder (BART, T5) models.

There are great resources for learning about the Transformer:

- [Illustrated Transformer](#) and [Illustrated GPT-2](#): very nice visual description of the Transformer.
- [Annotated Transformer](#): Pytorch implementation of the Transformer.

You are highly encouraged to read these references. In this lecture, I will strive to take a middle path which emphasizes pseudocode functions and interfaces.

The crux of the Transformers are the **attention mechanism**, which was developed earlier for machine translation ([Bahdananu et al. 2017](#)).

One can think of attention as a “soft” lookup table, where we have a query y that we want to match against each element in a sequence $x_{1:L} = [x_1, \dots, x_L]$:

$$[x_1, \dots, x_L] \quad y$$

We can think of each x_i as representing a key-value pair via linear transformations:

$$(W_{\text{key}} x_i) : (W_{\text{value}} x_i)$$

CS324

and forming the query via another linear transformation:

$$W_{\text{query}}y.$$

The key and the query can be compared to give a score:

$$\text{score}_i = x_i^\top W_{\text{key}}^\top W_{\text{query}}y.$$

These scores can be exponentiated and normalized to form a probability distribution over the token positions $\{1, \dots, L\}$:

$$[\alpha_1, \dots, \alpha_L] = \text{softmax}([\text{score}_1, \dots, \text{score}_L]).$$

Then the final output is a weighted combination over the values:

$$\sum_{i=1}^L \alpha_i (W_{\text{value}}x_i).$$

We can write this all succinctly in matrix form:

def Attention($x_{1:L} : \mathbb{R}^{d \times L}, y : \mathbb{R}^d$) $\rightarrow \mathbb{R}^d$:

- *Process y by comparing it to each x_i .*
- Return $W_{\text{value}} x_{1:L} \text{softmax}(x_{1:L}^\top W_{\text{key}}^\top W_{\text{query}}y / \sqrt{d})$.

We can think of there as being multiple aspects (e.g., syntax, semantics) that we would want to match on. To accommodate this, we can simultaneously have multiple **attention heads** and simply combine their outputs.

def MultiHeadedAttention($x_{1:L} : \mathbb{R}^{d \times L}, y : \mathbb{R}^d$) $\rightarrow \mathbb{R}^d$:

- *Process y by comparing it to each x_i with respect to n_{heads} aspects.*

CS324

- Return $W_{\text{output}} \underbrace{[\text{Attention}(x_{1:L}, y), \dots, \text{Attention}(x_{1:L}, y)]}_{n_{\text{heads}} \text{ times}}.$

Self-attention layer. Now we will substitute each x_i in for y as the query argument to produce:

def SelfAttention($x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Compare each element x_i to each other element.*
- Return $[\text{Attention}(x_{1:L}, x_1), \dots, \text{Attention}(x_{1:L}, x_L)]$.

Feedforward layer. Self-attention allows all the tokens to “talk” to each other, whereas feedforward connections provide:

def FeedForward($x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Process each token independently.*
- For $i = 1, \dots, L$:
 - Compute $y_i = W_2 \max(W_1 x_i + b_1, 0) + b_2$.
- Return $[y_1, \dots, y_L]$.

Improving trainability. We’re almost done. We could in principle just take the FeedForward $\circ$ SelfAttention sequence model and iterate it 96 times to make GPT-3, but that network would be hard to optimize (for the same vanishing gradients problems that afflicted RNNs, now just along the depth direction). So we have to do two shenanigans to make sure that the network is trainable.

Residual connections. One trick from computer vision is residual connections (ResNet). Instead of applying some function f :

$$f(x_{1:L}),$$

CS324

we add a residual (skip) connection so that if f 's gradients vanish, gradients can still flow through $x_{1:L}$:

$$x_{1:L} + f(x_{1:L}).$$

Layer normalization. Another trick is **layer normalization**, which takes a vector and makes sure its elements are too big:

def LayerNorm($x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Make each x_i not too big or small.*

We first define an adapter function that takes a sequence model f and makes it “robust”:

def AddNorm($f : (\mathbb{R}^{d \times L} \rightarrow \mathbb{R}^{d \times L})$, $x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Safely apply f to $x_{1:L}$.*
- Return LayerNorm($x_{1:L} + f(x_{1:L})$).

Finally, we can define the Transformer block succinctly as follows:

def TransformerBlock($x_{1:L} : \mathbb{R}^{d \times L}$) $\rightarrow \mathbb{R}^{d \times L}$:

- *Process each element x_i in context.*
- Return AddNorm(FeedForward, AddNorm(SelfAttention, $x_{1:L}$)).

Positional embeddings. You might have noticed that as defined, the embedding of a token doesn't depend on where it occurs in the sequence, so **mouse** in both sentences would have the same embedding, which is not sensible.

[the, mouse, ate, the, cheese]

[the, cheese, ate, the, mouse]

CS324

To fix this, we add **positional information** into the embedding:

def EmbedTokenWithPosition($x_{1:L} : \mathbb{R}^{d \times L}$):

- *Add in positional information.*
- Define positional embeddings:
 - Even dimensions: $P_{i,2j} = \sin(i/10000^{2j/d_{\text{model}}})$
 - Odd dimensions: $P_{i,2j+1} = \cos(i/10000^{2j/d_{\text{model}}})$
- Return $[x_1 + P_1, \dots, x_L + P_L]$.

GPT-3. With all the pieces in place, we can now define roughly GPT-3 architecture in one line, just by stacking the Transformer block 96 times:

$$\text{GPT-3}(x_{1:L}) = \text{TransformerBlock}^{96}(\text{EmbedTokenWithPosition}(x_{1:L}))$$

Shape of the architecture (how the 175 billion parameters are allocated):

- Dimension of hidden state: $d_{\text{model}} = 12288$
- Dimension of the intermediate feed-forward layer: $d_{\text{ff}} = 4d_{\text{model}}$
- Number of heads: $n_{\text{heads}} = 96$
- Context length: $L = 2048$

These decisions are not necessarily optimal. [Levine et al. 2020](#) provide some theoretical justification, showing that the GPT-3 is too deep, which motivated the training of a deeper but wider Jurassic architecture.

There are important but detailed differences between different versions of Transformers:

CS324

- Layer normalization “post-norm” (original Transformers paper) versus pre-norm (GPT-2), which impacts training stability ([Davis et al. 2021](#)).
- Dropout is applied throughout to prevent overfitting.
- GPT-3 uses a [sparse Transformer](#) to reduce the number of parameters, interleaving it with dense layers.
- Depending on the type of Transformer (encoder-only, decoder-only, encoder-decoder), different masking operations are used.
- And of course there are many more details involved in the training of Transformer models which we will discuss next time.

Further reading

Tokenization:

- [Between words and characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP](#). *Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, Samson Tan*. 2021. Comprehensive survey of tokenization.
- [Neural Machine Translation of Rare Words with Subword Units](#). *Rico Sennrich, B. Haddow, Alexandra Birch*. ACL 2015. Introduces **byte pair encoding** into NLP. Used by GPT-2, GPT-3.
- [Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation](#). *Yonghui Wu, M. Schuster, Z. Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, M. Krikun, Yuan Cao, Qin Gao, Klaus Macherey, J. Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Y. Kato, Taku Kudo, H. Kazawa, K. Stevens, George Kurian, Nishant Patil, W. Wang, C. Young, Jason R. Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, G. Corrado, Macduff Hughes, J. Dean*. 2016. Introduces **WordPiece**. Used by BERT.

CS324

- **SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing.** *Taku Kudo, John Richardson.* EMNLP 2018. Introduces **SentencePiece**.
-

Modeling:

- **Language Models are Unsupervised Multitask Learners.** Introduces GPT-2.
- **Attention is All you Need.** *Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin.* NIPS 2017.
- **Illustrated Transformer**
- **CS224N slides on RNNs**
- **CS224N slides on Transformers**
- **Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation.** *Ofir Press, Noah A. Smith, M. Lewis.* 2021. Introduces **Alibi embeddings**.
- **Transformer-XL: Attentive Language Models beyond a Fixed-Length Context.** *Zihang Dai, Zhilin Yang, Yiming Yang, J. Carbonell, Quoc V. Le, R. Salakhutdinov.* ACL 2019. Introduces recurrence on Transformers, relative position encoding scheme.
- **Generating Long Sequences with Sparse Transformers.** *R. Child, Scott Gray, Alec Radford, Ilya Sutskever.* 2019. Introduces **Sparse Transformers**.
- **Linformer: Self-Attention with Linear Complexity.** *Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, Hao Ma.* 2020. Introduces **Linformers**.
- **Rethinking Attention with Performers.** *K. Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, Adrian Weller.* ICLR 2020. Introduces **Performers**.

CS324

- **Efficient Transformers: A Survey.** *Yi Tay, M. Dehghani, Dara Bahri, Donald Metzler.* 2020.

 Decoder-only architectures:

- **Language Models are Unsupervised Multitask Learners.** *Alec Radford, Jeff Wu, R. Child, D. Luan, Dario Amodei, Ilya Sutskever.* 2019. Introduces **GPT-2** from OpenAI.
- **Language Models are Few-Shot Learners.** *Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, J. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. Henighan, R. Child, A. Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei.* NeurIPS 2020. Introduces **GPT-3** from OpenAI.
- **Scaling Language Models: Methods, Analysis&Insights from Training Gopher.** *Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, J. Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, G. V. D. Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John F. J. Mellor, I. Higgins, Antonia Creswell, Nathan McAleese, Amy Wu, Erich Elsen, Siddhant M. Jayakumar, Elena Buchatskaya, D. Budden, Esme Sutherland, K. Simonyan, Michela Paganini, L. Sifre, Lena Martens, Xiang Lorraine Li, A. Kuncoro, Aida Nematzadeh, E. Gribovskaya, Domenic Donato, Angeliki Lazaridou, A. Mensch, J. Lespiau, Maria Tsimpoukelli, N. Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Tobias Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d'Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, I. Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake A. Hechtman, Laura Weidinger, Iason Gabriel, William S. Isaac, Edward Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals,*

CS324

Kareem W. Ayoub, Jeff Stanway, L. Bennett, D. Hassabis, K. Kavukcuoglu, Geoffrey Irving. 2021. Introduces **Gopher** from DeepMind.

-
- **Jurassic-1: Technical details and evaluation.** *Opher Lieber, Or Sharir, Barak Lenz, Yoav Shoham.* 2021. Introduces **Jurassic** from AI21 Labs.

Encoder-only architectures:

- **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.** *Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova.* NAACL 2019. Introduces **BERT** from Google.
- **RoBERTa: A Robustly Optimized BERT Pretraining Approach.** *Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, M. Lewis, Luke Zettlemoyer, Veselin Stoyanov.* 2019. Introduces **RoBERTa** from Facebook.

Encoder-decoder architectures:

- **BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.** *M. Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, Luke Zettlemoyer.* ACL 2019. Introduces **BART** from Facebook.
- **Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.** *Colin Raffel, Noam M. Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, W. Li, Peter J. Liu.* J. Mach. Learn. Res. 2019. Introduces **T5** from Google.